```
    strtype s1("This is a test."), s2("I like C++");

    s1.show();
    s2.show();

    return 0;
}
```

REVIEW SKILLS CHECK: Chapter 5

1. A reference is a special type of pointer that is automatically dereferenced and that can be used interchangeably with the object it is pointing to. There are three types of references: parameter references, independent reference, and references that are returned by functions.

2. ```
   #include <iostream>
   using namespace std;

   int main()
   {
       float *f;
       int *i;

       f = new float;
       i = new int;

       if(!f || !i)
       {
           cout << "Allocation error\n";
           return 1;
       }

       *f = 10.101;
       *i = 100;

       cout << *f << ' ' << *i << '\n';

       delete f;
       delete i;

       return 0;
   }
   ```

3. The general form of **new** that includes an initializer is shown here:

   ```
   p_var = new type (initializer);
   ```

   For example, this allocates an integer and gives it the value 10:

   ```
   int *p;

   p = new int (10);
   ```
   ```

4.
```
#include <iostream>
using namespace std;

class samp
{
    int x;
public:
    samp(int n) { x = n; }
    int getx() { return x; }
};

int main()
{
    samp A[10] = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
    int i;

    for(i=0; i<10; i++)
        cout << A[i].getx() << ' ';

    cout << '\n';

    return 0;
}
```

5. Advantages: A reference parameter does not cause a copy of the object used in the call to be made. A reference is often faster to pass than a value. The reference parameter streamlines the call-by-reference syntax and procedure, reducing the chance for errors.

   Disadvantages: Changes to a reference parameter open the possibility of side effects in the calling routine.

6. No.

7.
```
#include <iostream>
using namespace std;

void mag(long &num, long order);

int main()
{
    long n = 4;
    long o = 2;

    cout << "4 raised to the 2nd order of magnitude is ";
    mag(n, o);
    cout <<  n << '\n';

    return 0;
}

void mag(long &num, long order)
{
    for( ; order; order--)
```

```
        num = num * 10;
}
```

5.1 EXERCISES

1. ```
   #include <iostream>
   #include <cstring>
   #include <cstdlib>
   using namespace std;

   class strtype
   {
       char *p;
       int len;
   public:
       strtype();
       strtype(char *s, int l);
       char *getstring() { return p; }
       int getlength() { return len; }
   };

   strtype::strtype()
   {
       p = new char [255];
       if(!p)
       {
           cout << "Allocation error\n";
           exit(1);
       }
       *p = '\0'; // null string
       len = 255;
   }

   strtype::strtype(char *s, int l)
   {
       if(strlen(s) >= l)
       {
           cout << "Allocating too little memory!\n";
           exit(1);
       }

       p = new char [l];
       if(!p)
       {
           cout << "Allocation error\n";
           exit(1);
       }
       strcpy(p, s);
       len = l;
   }
   ```
   ```

```
int main()
{
    strtype s1;
    strtype s2("This is a test", 100);

    cout << "s1: " << s1.getstring() << " - Length: ";
    cout << s1.getlength() << '\n';

    cout << "s2: " << s2.getstring() << " - Length: ";
    cout << s2.getlength() << '\n';

    return 0;
}
```

2.
```
// Stopwatch emulator
#include <iostream>
#include <ctime>
using namespace std;

class stopwatch
{
    double begin, end;
public:
    stopwatch();
    stopwatch(clock_t t);
    ~stopwatch();
    void start();
    void stop();
    void show();
};

stopwatch::stopwatch()
{
    begin = end = 0.0;
}

stopwatch::stopwatch(clock_t t)
{
    begin = (double) t / CLOCKS_PER_SEC;
    end = 0.0;
}

stopwatch::~stopwatch()
{
    cout << "Stopwatch object being destroyed...";
    show();
}

void stopwatch::start()
{
    begin = (double) clock() / CLOCKS_PER_SEC;
```

```
}

void stopwatch::stop()
{
    end = (double) clock() / CLOCKS_PER_SEC;
}

void stopwatch::show()
{
    cout << "Elapsed time: " << end - begin;
    cout << "\n";
}

int main()
{
    stopwatch watch;
    long i;

    watch.start();
    for(i=0; i<320000; i++); // time a for loop
    watch.stop();
    watch.show();

    // create object using initial value
    stopwatch s2(clock());
    for(i=0; i<250000; i++); // time a for loop
    s2.stop();
    s2.show();

    return 0;
}
```

5.2 EXERCISES

1. The **obj** and **temp** objects are constructed normally. However, when **temp** is returned by **f()**, a temporary object is made, and it is this temporary object that generates the call to the copy constructor.

2. As the program is written, when an object is passed to **getval()** a bitwise copy is made. When **getval()** returns and that copy is destroyed, the memory allocated to that object (which is pointed to by **p**) is released. However, this is the same memory still required by the object used in the call to **getval()**. The correct version of the program is shown here. It uses a copy constructor to avoid this problem.

```
// This program is now fixed.
#include <iostream>
#include <cstdlib>
using namespace std;

class myclass
{
    int *p;
```

```
public:
    myclass(int i);
    myclass(const myclass &o); // copy constructor
    ~myclass() { delete p; }
    friend int getval(myclass o);
};

myclass::myclass(int i)
{
    p = new int;

    if(!p)
    {
        cout << "Allocation error\n";
        exit(1);
    }
    *p = i;
}

// Copy constructor
myclass::myclass(const myclass &o)
{
    p = new int; // allocate copy's own memory

    if(!p)
    {
        cout << "Allocation error\n";
        exit(1);
    }
    *p = *o.p;
}

int getval(myclass o)
{
    return *o.p; // get value
}

int main()
{
    myclass a(1), b(2);

    cout << getval(a) << " " << getval(b);
    cout << "\n";
    cout << getval(a) << " " << getval(b);

    return 0;
}
```

3. A copy constructor is invoked when one object is used to initialize another. A normal constructor is called when an object is created.

5.4 EXERCISES

1. ```
   #include <iostream>
   #include <cstdlib>
   using namespace std;

   long mystrtol(const char *s, char **end, int base = 10)
   {
       return strtol(s, end, base);
   }

   int main()
   {
       long x;
       char *s1 = "100234";
       char *p;

       x = mystrtol(s1, &p, 16);
       cout << "Base 16: " << x << '\n';

       x = mystrtol(s1, &p, 10);
       cout << "Base 10: " << x << '\n';

       x = mystrtol(s1, &p); // use default base of 10
       cout << "Base 10: " << x << '\n';

       return 0;
   }
   ```

2. All parameters taking default arguments must appear to the right of those that do not. That is, once you begin giving parameters defaults, all subsequent parameters must also have defaults. In the question, **q** is not given a default.

3. Since cursor positioning functions differ from compiler to compiler and environment to environment, only one possible solution is shown. The following program works for Borland C++ under a command-prompt environment.

   ```
   // Note: This program is Borland C++-specific.
   #include <iostream>
   #include <conio.h>
   using namespace std;

   void myclreol(int len = -1);

   int main()
   {
       int i;

       gotoxy(1, 1);
       for(i=0; i<24; i++)
           cout << "abcdefghijklmnopqrstyvwxyz1234567890\n";
   ```
   ```

```
    gotoxy(1, 2);
    myclreol();
    gotoxy(1, 4);
    myclreol(20);

    return 0;
}

// Clear to end of line unless len parameter is specified.
void myclreol(int len)
{
    int x, y;

    x = wherex(); // get x position
    y = wherey(); // get y position

    if(len == -1)
        len = 80-x;

    int i = x;

    for( ; i<=len; i++)
        cout << ' ';

    gotoxy(x, y); // reset the cursor
}
```

4. A default argument cannot be another parameter or a local variable.

5.6 EXERCISE

1.
```
#include <iostream>
using namespace std;

int dif(int a, int b)
{
    return a-b;
}

float dif(float a, float b)
{
    return a-b;
}

int main()
{
    int (*p1)(int, int);
    float (*p2)(float, float);

    p1 = dif; // address of dif(int, int)
    p2 = dif; // address of dif(float, float)
```

```
    cout << p1(10, 5) << ' ';
    cout << p2(10.5, 8.9) << '\n';

    return 0;
}
```

MASTERY SKILLS CHECK: Chapter 5

1.
```
// Overload date() for time_t.
#include <iostream>
#include <cstdio> // included for sscanf()
#include <ctime>
using namespace std;

class date
{
    int day, month, year;
public:
    date(char *str);
    date(int m, int d, int y)
    {
        day = d;
        month = m;
        year = y;
    }
    // overload for parameter of type time_t
    date(time_t t);
    void show()
    {
        cout << month << '/' << day << '/';
        cout << year << '\n';
    }
};

date::date(char *str)
{
    sscanf(str, "%d%*c%d%*c%d", &month, &day, &year);
}

date::date(time_t t)
{
    struct tm *p;

    p = localtime(&t); // convert to broken down time
    day = p->tm_mday;
    month = p->tm_mon;
    year = p->tm_year;
}

int main()
{
```

```
    // construct date object using string
    date sdate("12/31/99");

    // construct date object using integers
    date idate(12, 31, 99);

    /*
       construct date object using time_t - this
       creates an object using the system date
    */
    date tdate(time(NULL));

    sdate.show();
    idate.show();
    tdate.show();

    return 0;
}
```

2. The class **samp** defines only one constructor, and this constructor requires an initializer. Therefore, it is improper to declare an object of type **samp** without one. (That is, **samp x** is an invalid declaration.)

3. One reason to overload a constructor is to provide flexibility, allowing you to choose the most appropriate constructor in the specific instance. Another is to allow both initialized and uninitialized objects to be declared. You might want to overload a constructor so that dynamic arrays can be allocated.

4. The most common general form of a copy constructor is shown here:

```
classname (const classname &obj)
{
    // body of constructor
}
```

5. A copy constructor is called when an initialization takes place-specifically, when one object is explicitly used to initialize another, when an object is passed as a parameter to a function, and when a temporary object is created when an object is returned by a function.

6. The **overload** keyword is obsolete. In early versions of C++ it was used to tell the compiler that a function will be overloaded. It is not supported by modern compilers.

7. A default argument is a value that is given to a function parameter when no corresponding argument appears when the function is called.

8.
```
#include <iostream>
#include <cstring>
using namespace std;

void reverse(char *str, int count  = 0);

int main()
{
    char s1[] = "This is a test.";
```

```
    char s2[] = "I like C++.";

    reverse(s1); // reverses entire string
    reverse(s2, 7); // reverse first 7 chars

    cout << s1 << '\n';
    cout << s2 << '\n';

    return 0;
}

void reverse(char *str, int count)
{
    int i, j;
    char temp;

    if(!count)
        count = strlen(str) - 1;
    for(i=0, j=count; i<j; i++, j--)
    {
        temp = str[i];
        str[i] = str[j];
        str[j] = temp;
    }
}
```

9. All parameters receiving default arguments must appear to the right of those that do not.

10. Ambiguity can be introduced by default type conversions, reference parameters, and default arguments.

11. It is ambiguous because the compiler cannot know which version of **compute()** to call. Is it the first version, with **divisor** defaulting? Or is it the second version, which takes only one parameter?

12. When you are obtaining the address of an overloaded function, it is the type specification of the pointer that determines which function is used.

CUMULATIVE SKILLS CHECK: Chapter 5

1.
```
#include <iostream>
using namespace std;

void order(int &a, int &b)
{
    int t;

    if(a<b)
        return;
    else // swap a and b
    {
        t = a;
```

```
        a = b;
        b = t;
    }
}

int main()
{
    int x=10, y=5;

    cout << "x: " << x << ", y: " << y << '\n';

    order(x, y);
    cout << "x: " << x << ", y: " << y << '\n';

    return 0;
}
```

2. The syntax for calling a function that takes a reference parameter is identical to the syntax for calling a function that takes a value parameter.

3. A default argument is essentially a shorthand approach to function overloading because the net result is the same.
   For example,

   ```
   int f(int a, int b = 0);
   ```

   is functionally equivalent to these two overloaded functions:

   ```
   int f(int a);

   int f(int a, int b);
   ```

4. ```
   #include <iostream>
   using namespace std;

   class samp
   {
       int a;
   public:
       samp() { a = 0; }
       samp(int n) { a = n; }
       int get_a() { return a; }
   };

   int main()
   {
       samp ob(88);
       samp obarray[10];

       // ...
   }
   ```
   ```

5. Copy constructors are needed when you, the programmer, must control precisely how a copy of an object is made. This is important only when the default bitwise copy creates undesired side effects.

REVIEW SKILLS CHECK: Chapter 6

1.
```
class myclass
{
    int x, y;
public:
    myclass(int i, int j) { x=i; y=j; }
    myclass() { x=0; y=0; }
};
```

2.
```
class myclass
{
    int x, y;
public:
    myclass(int i=0, int j=0) { x=i; y=j; }
};
```

3. Once default arguments have begun, a non-defaulting parameter cannot occur.

4. A function cannot be overloaded when the only difference is that one version takes a value parameter and the other takes a reference parameter. (The compiler cannot tell them apart.)

5. It is appropriate to use default arguments when there are one or more values that will occur frequently. Is is inappropriate when there is no value or values that have a greater likelihood of occurring.

6. No, because there is no way to initialize a dynamic array. This class has only one constructor, and it requires initializers.

7. A copy constructor is a special constructor that is called when one object initializes another. This circumstance can occur in any of the following three ways: When one object is explicitly used to initialize another, when an object is passed to a function, or when a temporary object is created as a function return value.

6.2 EXERCISES

1.
```
// Overload the * and / relative to coord class.
#include <iostream>
using namespace std;

class coord
{
    int x, y; // coordinate values
public:
    coord() { x=0; y=0; }
    coord(int i, int j) { x=i; y=j; }
    void get_xy(int &i, int &j) { i=x; j=y; }
```